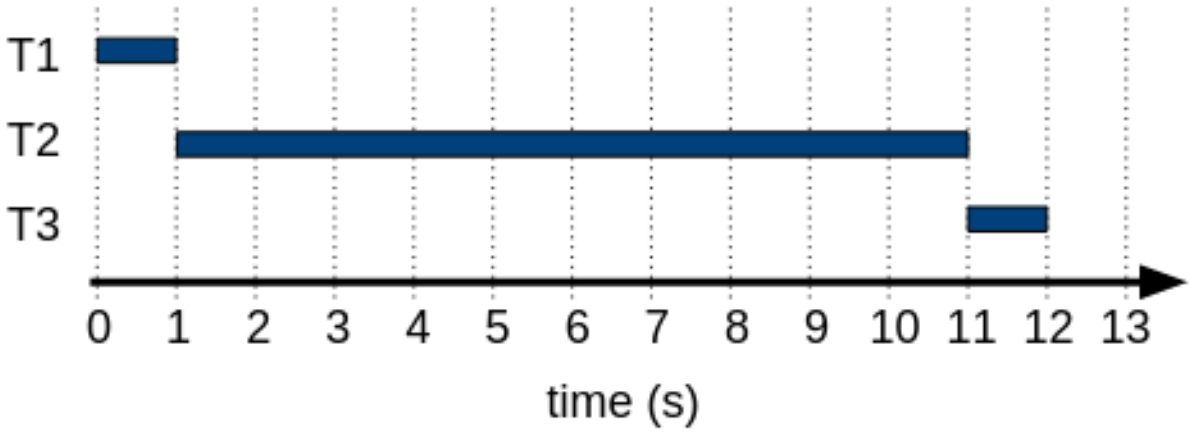
“Bare-Metal Embedded Systems with AVR128DA48 Course”

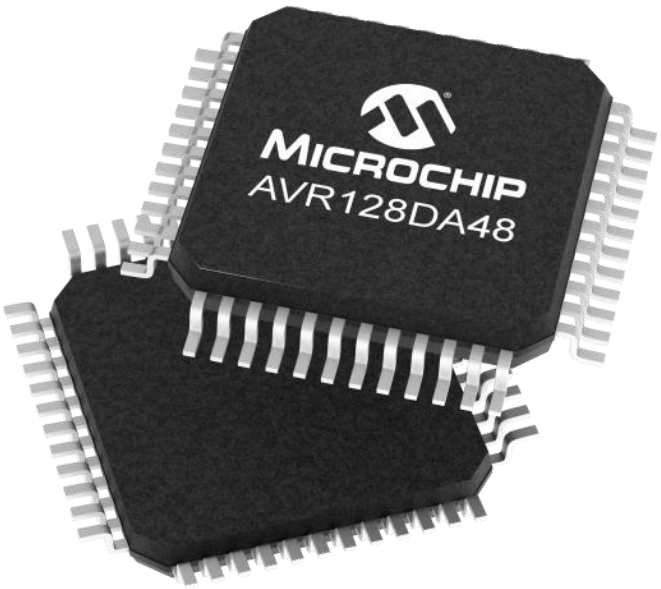
Day 29 - Cooperative Task Scheduling

name:	T1
arrive:	@ 0s
duration:	1s

name:	T2
arrive:	@ 0.1s
duration:	10s

name:	T3
arrive:	@ 0.2s
duration:	1s





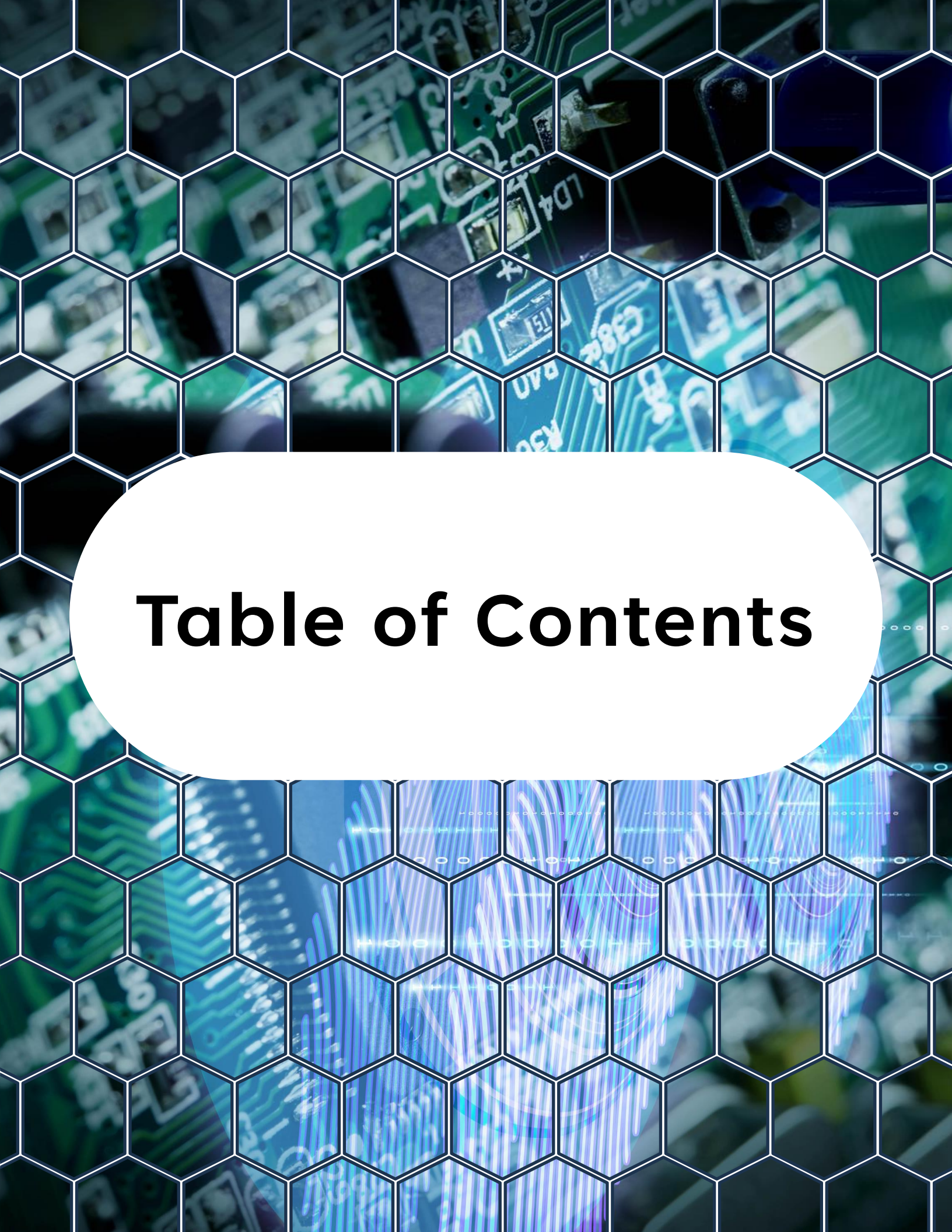
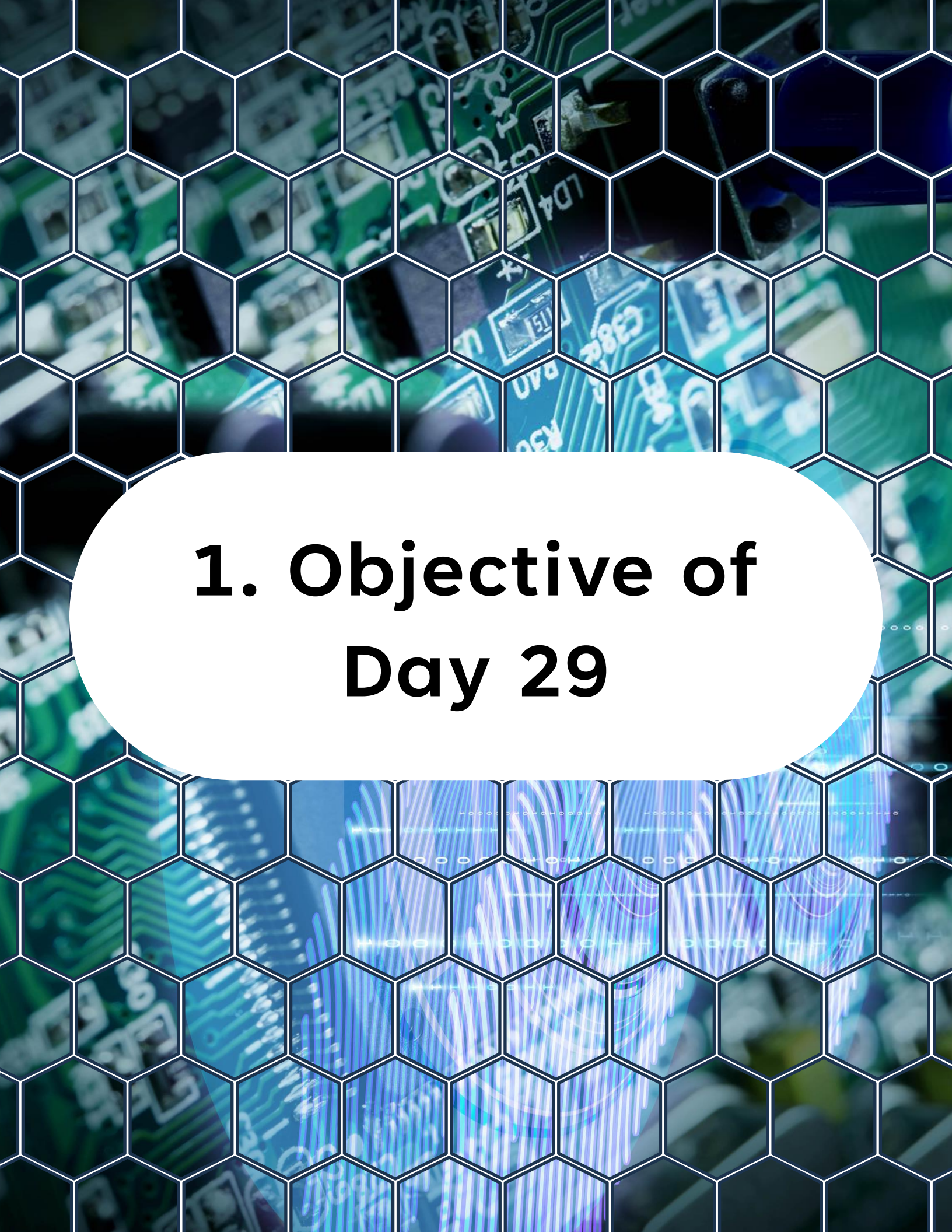


Table of Contents

Table of Contents

Table of Contents



1. Objective of Day 29

1. Objective of Day 29

So far, we have written firmware in a mostly direct style:

- Initialize peripherals
- Enter while (1)
- Poll inputs
- React to events

That works for small programs.

But as your firmware grows, a single unstructured loop quickly becomes difficult to maintain. Different tasks start competing for CPU time. Timing becomes inconsistent. Delays in one part of the code affect everything else.

1. Objective of Day 29

Today we study a simple and very practical solution:

Cooperative Task Scheduling

By the end of today, you will understand:

- What a super-loop architecture is
- How cooperative scheduling works
- How to use timers for time slicing
- How state machines fit naturally into scheduled firmware
- How to design a clean non-blocking main loop

This is one of the most important steps between small examples and real embedded firmware.

2. What Is Cooperative Task Scheduling?

2. What Is Cooperative Task Scheduling?

Cooperative scheduling is a way to organize firmware so that multiple tasks share CPU time in a controlled manner.

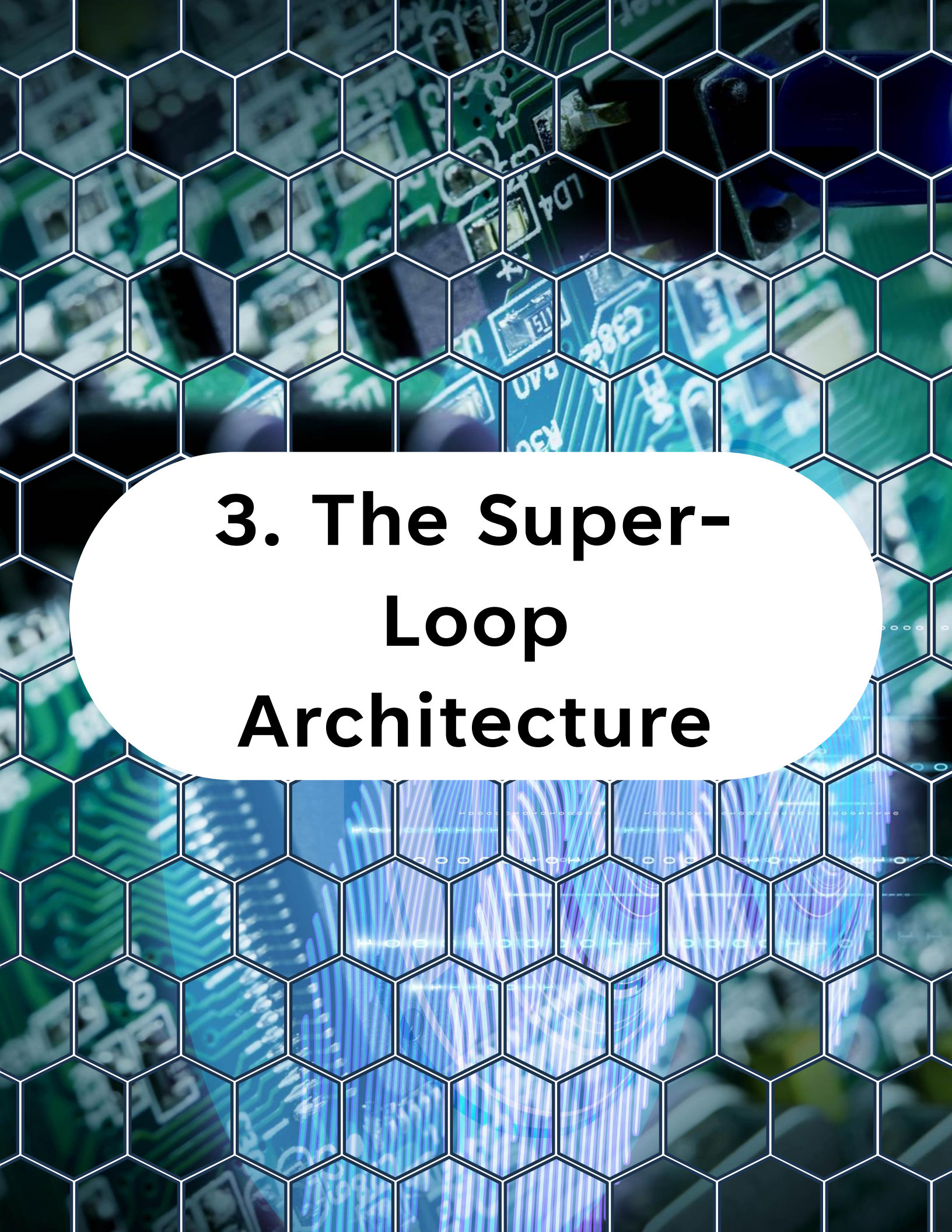
Each task:

- Runs briefly
- Does only a small amount of work
- Returns control quickly

No task is forcibly interrupted by a scheduler.

Instead, each task cooperates.

That is why it is called cooperative scheduling.



3. The Super-Loop Architecture

3. The Super-Loop Architecture

The simplest form of cooperative scheduling is the super-loop.

Basic structure:

```
while (1)
{
    task_A();
    task_B();
    task_C();
}
```

This loop runs forever.

Each task gets CPU time because the loop keeps cycling.

3. The Super-Loop Architecture

3.1 Why the Super-Loop Is Useful

The super-loop is:

- Simple
- Predictable
- Easy to debug
- Small in code size
- Ideal for bare-metal systems

It is often the first real scheduler used in embedded products.

3.2 The Problem with Naive Super-Loops

A raw super-loop becomes problematic if tasks contain:

- Long delays

3. The Super-Loop Architecture

- Blocking waits
- Busy loops
- Large computations

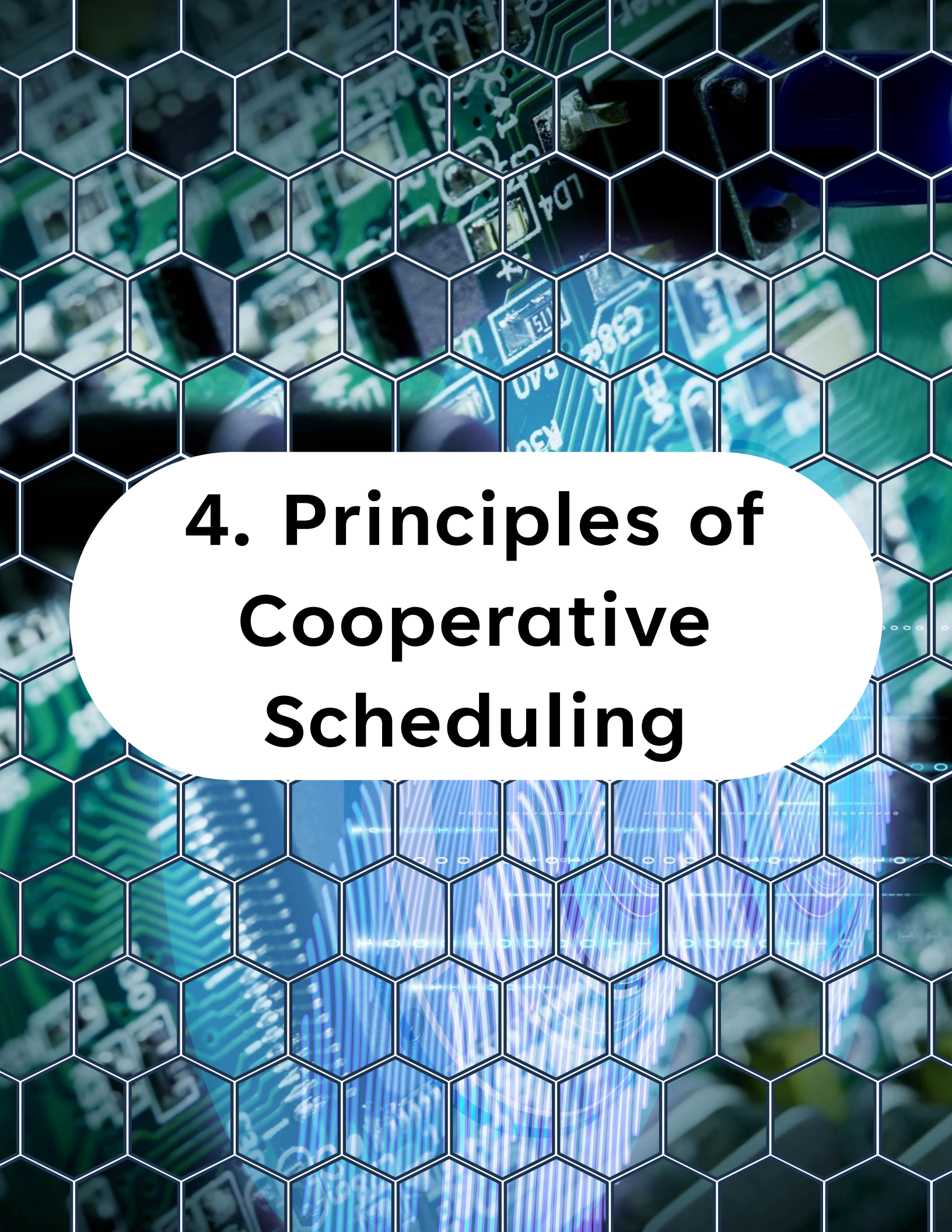
Example of bad design:

```
while (1)
{
    read_sensor();
    delay_ms(500);
    update_display();
    delay_ms(500);
}
```

This is not real scheduling.

It blocks the entire system.

If one task waits, all other tasks wait too.



4. Principles of Cooperative Scheduling

4. Principles of Cooperative Scheduling

A cooperative scheduler works well only if tasks follow a few important rules.

4.1 Keep Tasks Short

A task should execute quickly.

Good task:

- Check a flag
- Update a variable
- Start a peripheral operation
- Return

Bad task:

- Sit in long delay loops
- Wait for slow operations
- Block until something happens

4. Principles of Cooperative Scheduling

4.2 Never Block in a Task

This is the most important rule.

Do not do this:

```
while (!(ADC0.INTFLAGS & ADC_RESRDY_bm))
{
}
```

inside a scheduled task unless the wait is guaranteed to be extremely short and controlled.

Instead:

- Start work in one pass
- Check completion in later passes

That keeps the system responsive.

4. Principles of Cooperative Scheduling

4.3 Use Flags and State Machines

Tasks should usually operate as:

small decision blocks

flag handlers

state machines

This makes them cooperative by design.

5. Time Slicing with Timers

5. Time Slicing with Timers

A cooperative system often needs tasks to run at different rates.

Example:

- LED task every 100 ms
- Sensor task every 10 ms
- Status task every 1 s

We do this by using a hardware timer to generate a periodic time base.

That time base is often called a:

- system tick
- scheduler tick
- time slice base

5. Time Slicing with Timers

5.1 Using a Timer as the System Tick

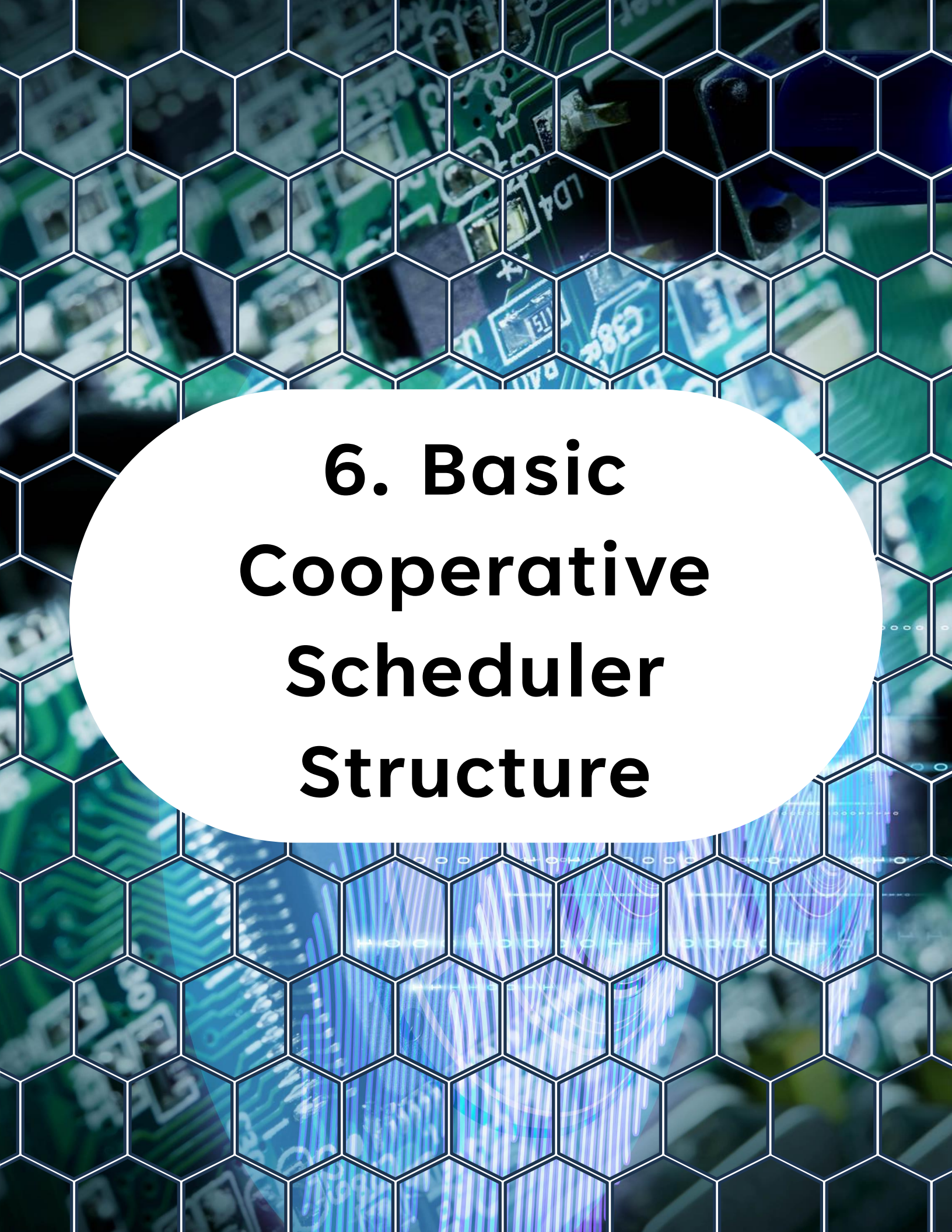
We can configure a timer to interrupt every 1 ms.

Inside the ISR:

- increment a tick counter
- set task flags at the required intervals

Then the main loop checks those flags and runs tasks.

This gives us deterministic scheduling without blocking delays.



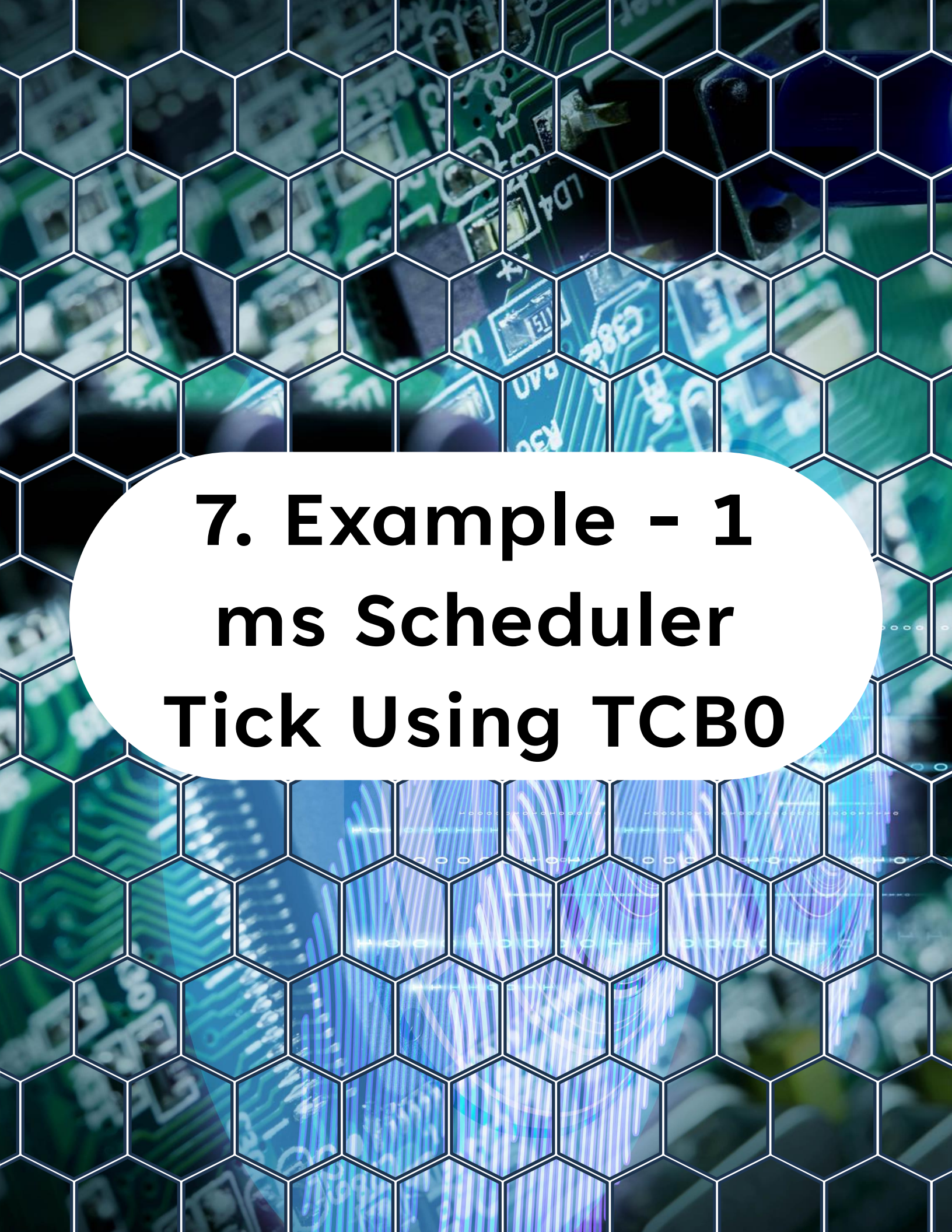
6. Basic Cooperative Scheduler Structure

6. Basic Cooperative Scheduler Structure

The architecture looks like this:

1. Timer interrupt creates time base
2. ISR sets task flags
3. Main loop checks task flags
4. Corresponding tasks run
5. Tasks return immediately

This is simple, reliable, and very common in bare-metal systems.



7. Example – 1 ms Scheduler Tick Using TCB0

7. Example - 1 ms Scheduler Tick Using TCB0

Assume:

- F_CPU = 24 MHz
- TCB clock = CLK_PER / 2 = 12 MHz
- Want 1 ms interrupt

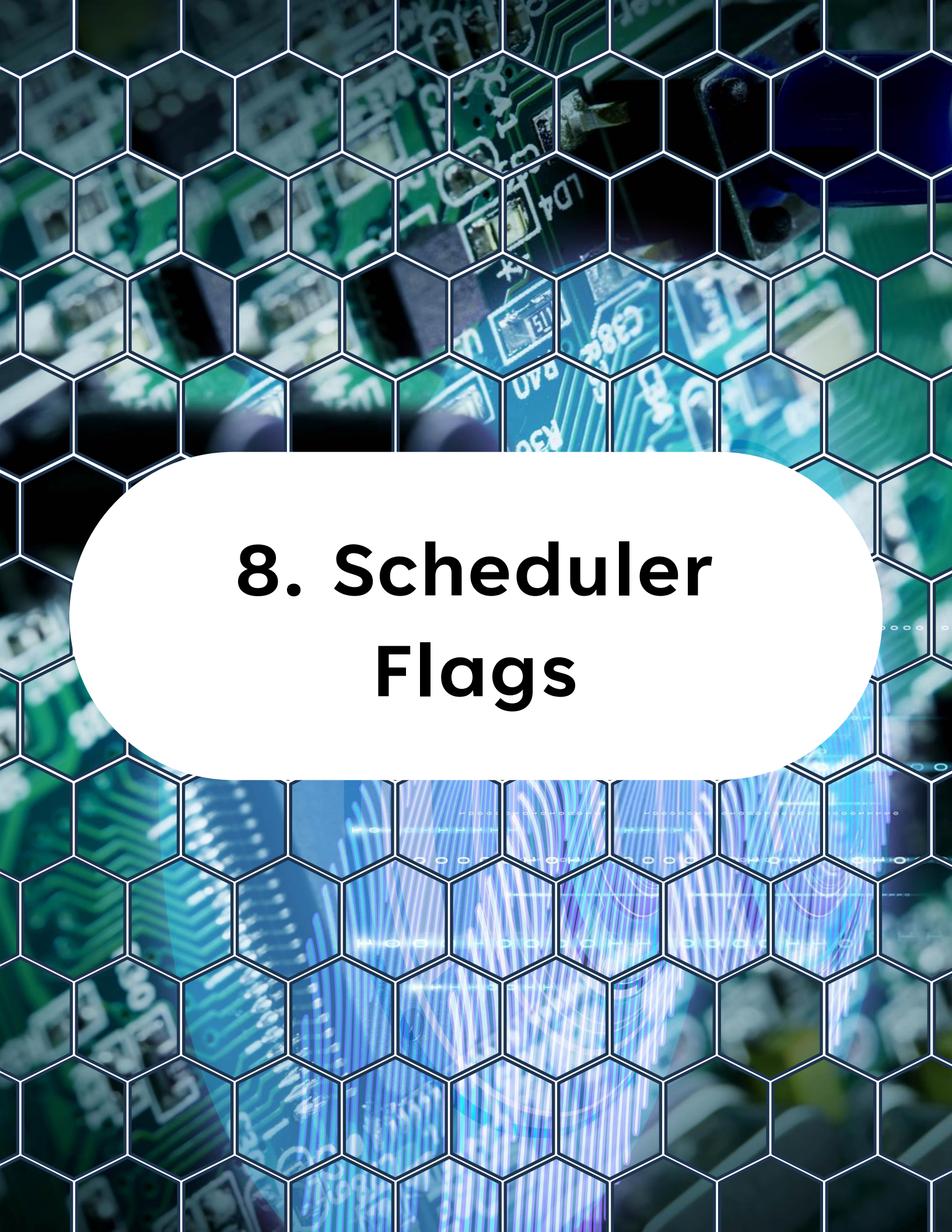
Then:

12000000 / 1000 = 12000

So TCB0.CCMP = 12000

7. Example - 1 ms Scheduler Tick Using TCB0

```c
/**
 * @brief Initialize TCB0 to generate a 1 ms scheduler tick.
 */
static void TCB0_init(void)
{
    // Disable TCB0 before configuration
    TCB0.CTRLA &= ~(1 << TCB_ENABLE_bp);

    // Periodic interrupt mode
    TCB0.CTRLB = TCB_CNTMODE_INT_gc;

    // Set compare value for 1 ms interval
    // 24 MHz / 2 = 12 MHz, 12 MHz / 12000 = 1 kHz (1 ms period)
    TCB0.CCMP = 12000;
    // Clear any pending interrupt flags
    TCB0.INTFLAGS = TCB_CAPT_bm;

    // Enable capture/compare interrupt
    TCB0.INTCTRL = TCB_CAPT_bm;

    // Enable timer, use CLK_PER divided by 2 for better resolution
    TCB0.CTRLA = TCB_CLKSEL_DIV2_gc | TCB_ENABLE_bm;
}
```



8. Scheduler Flags

8. Scheduler Flags

We use simple flags to mark tasks ready.

Example:

```
volatile uint8_t task_10ms_flag = 0;
volatile uint8_t task_100ms_flag = 0;
volatile uint8_t task_1000ms_flag = 0;
```

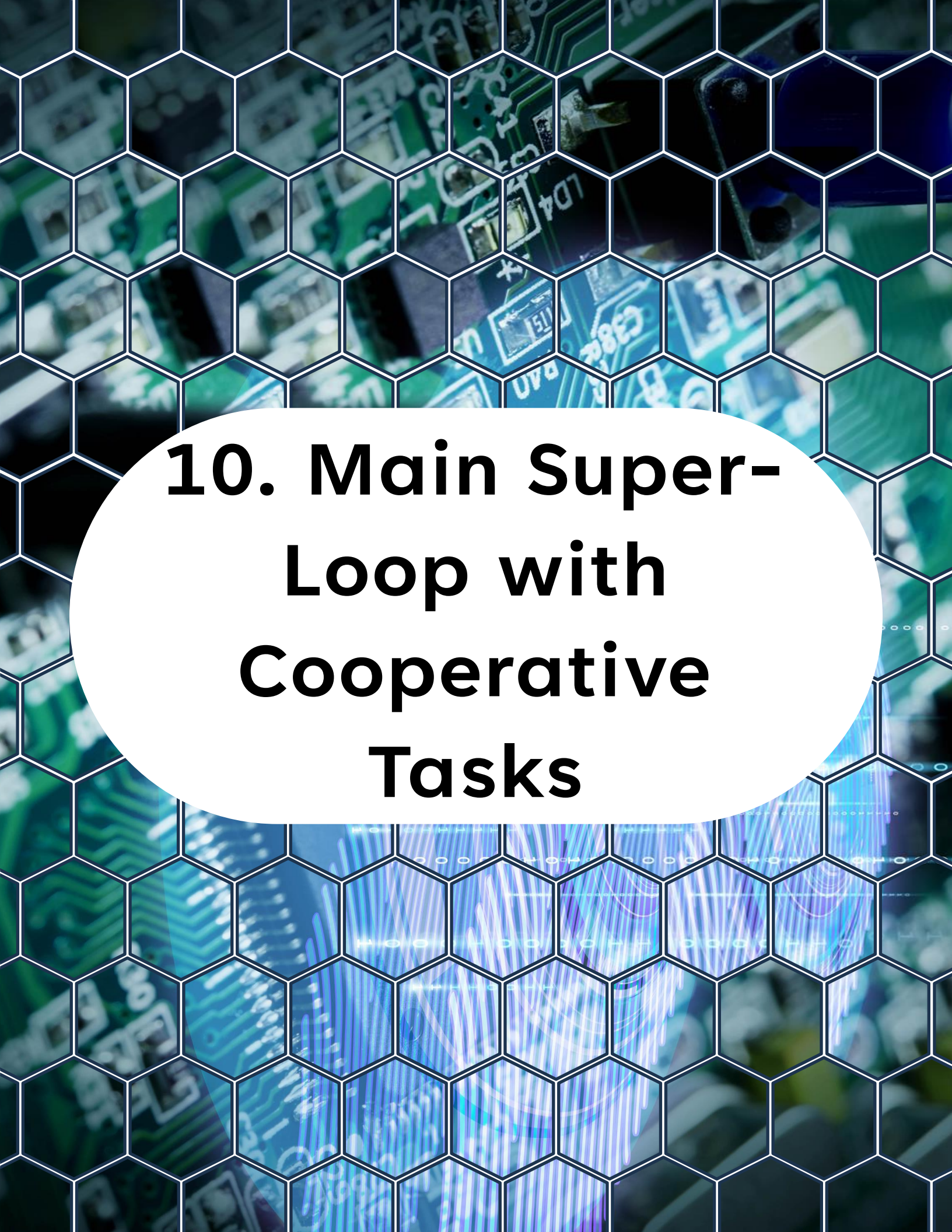
These flags are set in the timer ISR.

9. Timer ISR for Time Slicing

9. Timer ISR for Time Slicing

```c
#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdint.h>

volatile uint8_t task_10ms_flag = 0;
volatile uint8_t task_100ms_flag = 0;
volatile uint8_t task_1000ms_flag = 0;

ISR(TCB0_INT_vect) {
    static uint16_t tick_count = 0;

    TCB0.INTFLAGS = TCB_CAPT_bm;

    tick_count++;

    if ((tick_count % 10u) == 0u) {
        task_10ms_flag = 1u;
    }

    if ((tick_count % 100u) == 0u) {
        task_100ms_flag = 1u;
    }

    if ((tick_count % 1000u) == 0u) {
        task_1000ms_flag = 1u;
        tick_count = 0u;
    }
}
```

This gives us three software time slices.



10. Main Super-Loop with Cooperative Tasks

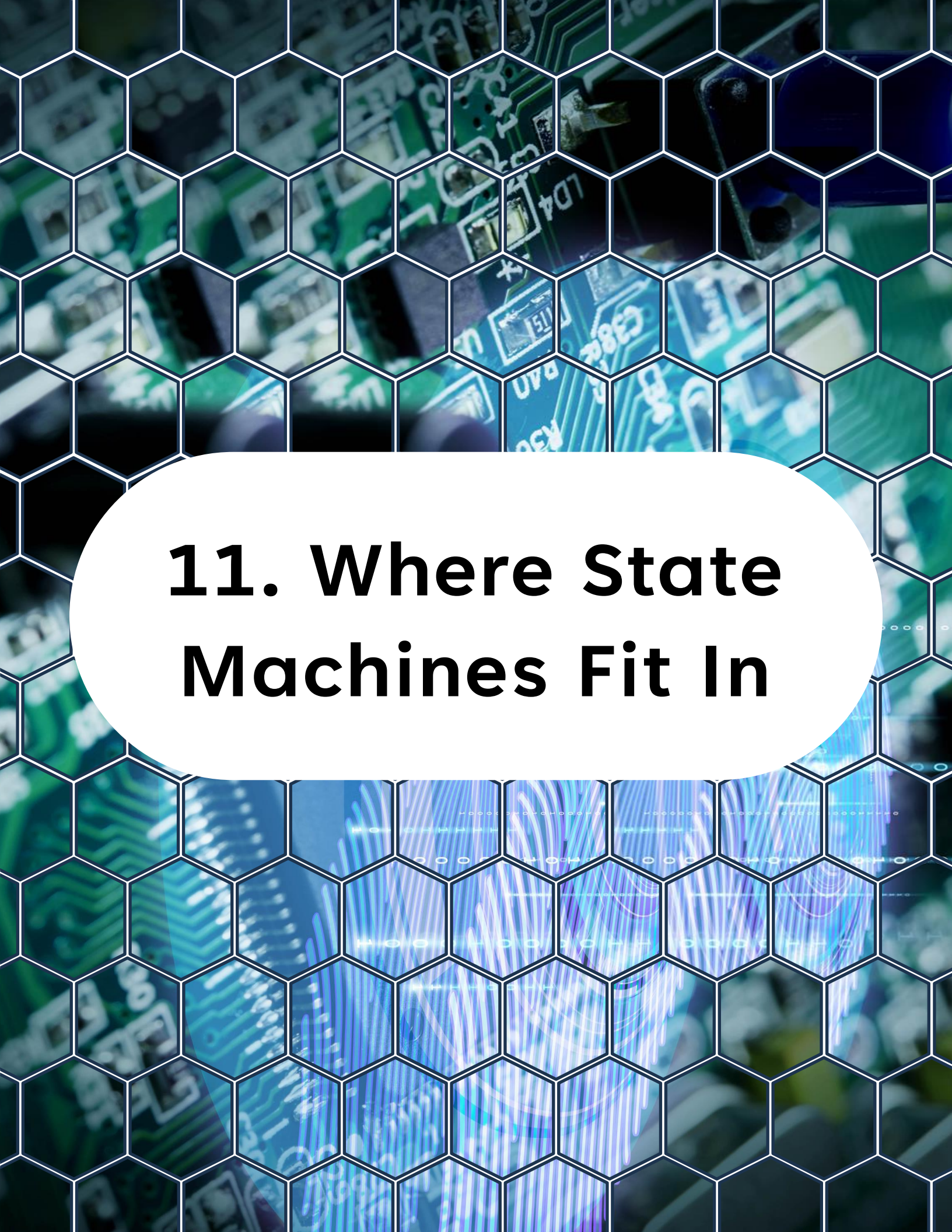
10. Main Super-Loop with Cooperative Tasks

Now the super-loop becomes structured:

```c
while (1)
{
    if (task_10ms_flag)
    {
        task_10ms_flag = 0u;
        task_10ms();
    }

    if (task_100ms_flag)
    {
        task_100ms_flag = 0u;
        task_100ms();
    }

    if (task_1000ms_flag)
    {
        task_1000ms_flag = 0u;
        task_1000ms();
    }
}
```

This is a real cooperative scheduler.

No blocking delays, No RTOS, No hidden behavior.



11. Where State Machines Fit In

11. Where State Machines Fit In

State machines are a perfect match for cooperative scheduling.

Why?

- Because a good state machine:
- Executes a small step
- Decides what happens next
- Returns immediately
- That is exactly what cooperative tasks need.

Instead of writing one huge task with blocking logic, you write a state machine that advances over time.

11. Where State Machines Fit In

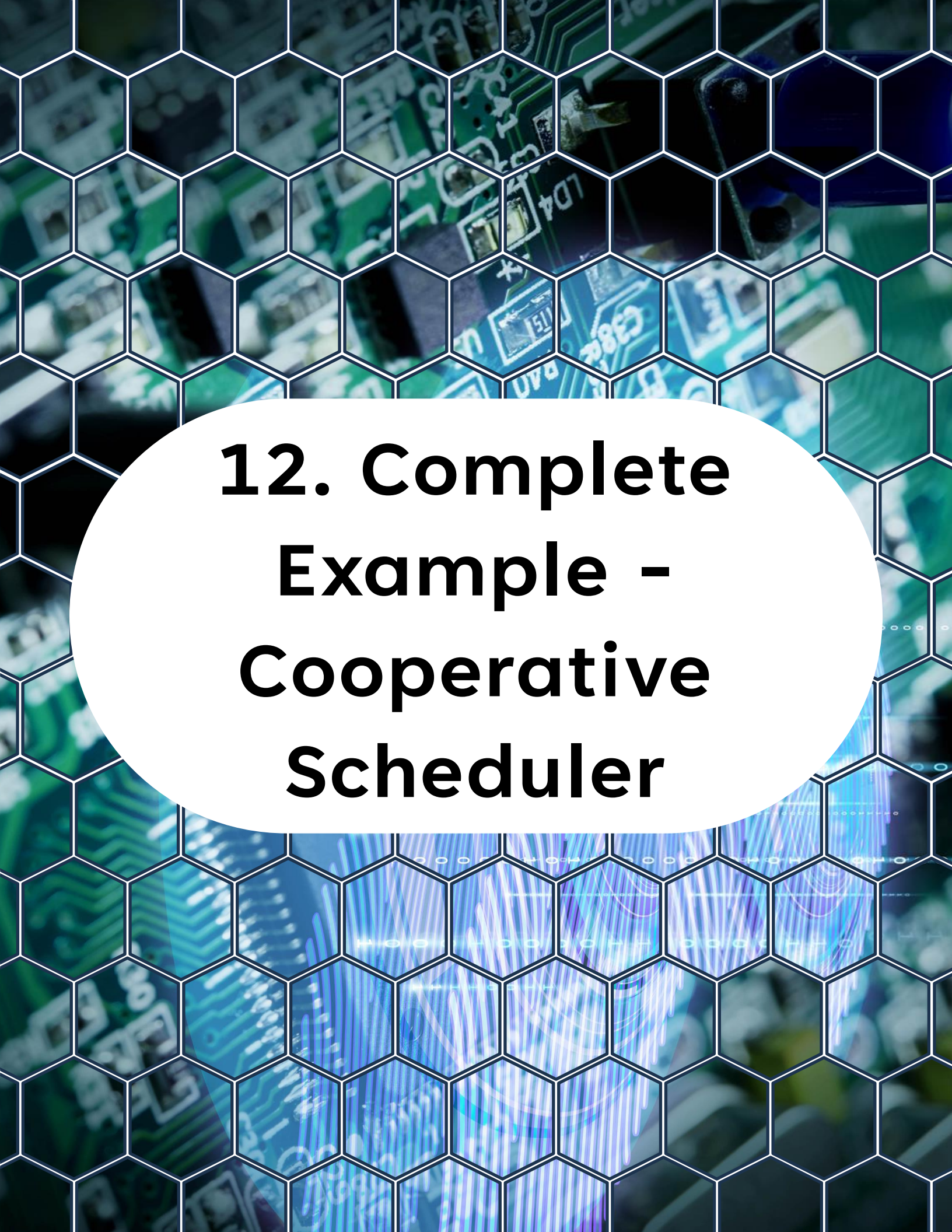
Example Concept

A sensor task may have these states:

- IDLE
- START_ADC
- WAIT_ADC
- PROCESS_RESULT

Each time the task runs, it advances one step if conditions are met.

This is much better than starting an ADC conversion and waiting in a loop.



12. Complete Example – Cooperative Scheduler

12. Complete Example - Cooperative Scheduler

This example demonstrates:

- 1 ms timer tick
- LED toggle every 100 ms
- ADC sampling task every 10 ms
- Status state machine every 1000 ms

```c
/**
 * @file main.c
 * @brief Cooperative task scheduling example using TCB0 time base.
 */

#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdint.h>

#define LED_PORT   PORTC
#define LED_PIN    PIN6_bm

volatile uint8_t task_10ms_flag = 0u;
volatile uint8_t task_100ms_flag = 0u;
volatile uint8_t task_1000ms_flag = 0u;

volatile uint16_t g_adc_value = 0u;

typedef enum
{
    STATUS_STATE_IDLE = 0,
    STATUS_STATE_SAMPLE,
```

12. Complete Example - Cooperative Scheduler

```
19  typedef enum
20  {
21      STATUS_STATE_IDLE = 0,
22      STATUS_STATE_SAMPLE,
23      STATUS_STATE_UPDATE
24  } status_state_t;
25
26  static status_state_t g_status_state = STATUS_STATE_IDLE;
27
28  /**
29   * @brief Initialize LED pin.
30   */
31  static void LED_init(void)
32  {
33      LED_PORT.DIRSET = LED_PIN;
34      LED_PORT.OUTCLR = LED_PIN;
35  }
36
37  /**
38   * @brief Toggle LED output.
39   */
40  static void LED_toggle(void)
41  {
42      LED_PORT.OUTTGL = LED_PIN;
43  }
44
45  /**
46   * @brief Initialize ADC0 for single-ended 12-bit conversion.
47   */
48  static void ADC0_init(void)
49  {
50      // Disable ADC before configuration
51      ADC0.CTRLA = 0;
52
53      // Select reference
54      // Enable reference always ON for ADC0
55      // Internal 4.096V reference
56      VREF.ADC0REF = 1 << VREF_ALWAYSON_bp
57                   | VREF_REFSEL_4V096_gc;
```

12. Complete Example - Cooperative Scheduler

```
53      // Select reference
54      // Enable reference always ON for ADC0
55      // Internal 4.096V reference
56      VREF.ADC0REF = 1 << VREF_ALWAYSON_bp
57                   | VREF_REFSEL_4V096_gc;
58
59      // ADC input pin 0
60      ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;
61
62      // No accumulation
63      ADC0.CTRLB = ADC_SAMPNUM_NONE_gc;
64
65      // Prescaler = CLK_PER / 16
66      ADC0.CTRLC = ADC_PRESC_DIV16_gc;
67
68      // 12-bit mode
69      ADC0.CTRLA |= ADC_RESSEL_12BIT_gc;
70
71      // Single-ended conversion mode
72      ADC0.CTRLA |= 0 << ADC_CONVMODE_bp;
73
74      // Enable ADC
75      ADC0.CTRLA |= 1 << ADC_ENABLE_bp;
76  }
77
78  /**
79   * @brief Start ADC conversion.
80   */
81  static void ADC0_start(void)
82  {
83      ADC0.COMMAND = ADC_STCONV_bm;
84  }
85
86  /**
87   * @brief Check whether ADC result is ready.
88   *
89   * @return 1 if ready, otherwise 0.
90   */
91  static uint8_t ADC0_ready(void)
```

12. Complete Example - Cooperative Scheduler

```
86  /**
87   * @brief Check whether ADC result is ready.
88   *
89   * @return 1 if ready, otherwise 0.
90   */
91  static uint8_t ADC0_ready(void)
92  {
93      return (ADC0.INTFLAGS & ADC_RESRDY_bm) ? 1u : 0u;
94  }
95
96  /**
97   * @brief Read ADC result.
98   *
99   * @return ADC conversion result.
100  */
101 static uint16_t ADC0_read(void)
102 {
103     ADC0.INTFLAGS = ADC_RESRDY_bm;
104     return ADC0.RES;
105 }
106
107 /**
108  * @brief Initialize TCB0 to generate a 1 ms scheduler tick.
109  */
110 static void TCB0_init(void)
111 {
112     // Disable TCB0 before configuration
113     TCB0.CTRLA &= ~(1 << TCB_ENABLE_bp);
114
115     // Periodic interrupt mode
116     TCB0.CTRLB = TCB_CNTMODE_INT_gc;
117
118     // Set compare value for 1 ms interval
119     // 24 MHz / 2 = 12 MHz, 12 MHz / 12000 = 1 kHz (1 ms period)
120     TCB0.CCMP = 12000;
121     // Clear any pending interrupt flags
122     TCB0.INTFLAGS = TCB_CAPT_bm;
123
```

12. Complete Example - Cooperative Scheduler

```
124     // Enable capture/compare interrupt
125     TCB0.INTCTRL = TCB_CAPT_bm;
126
127     // Enable timer, use CLK_PER divided by 2 for better resolution
128     TCB0.CTRLA = TCB_CLKSEL_DIV2_gc | TCB_ENABLE_bm;
129 }
130
131 /**
132  * @brief Task executed every 10 ms.
133  *
134  * Starts a new ADC conversion if previous one is complete.
135  */
136 static void task_10ms(void)
137 {
138     if (ADC0_ready())
139     {
140         g_adc_value = ADC0_read();
141     }
142
143     ADC0_start();
144 }
145
146 /**
147  * @brief Task executed every 100 ms.
148  *
149  * Toggles LED.
150  */
151 static void task_100ms(void)
152 {
153     LED_toggle();
154 }
155
156 /**
157  * @brief Task executed every 1000 ms.
158  *
159  * Demonstrates a simple cooperative state machine.
160  */
161 static void task_1000ms(void)
162 {
```

12. Complete Example - Cooperative Scheduler

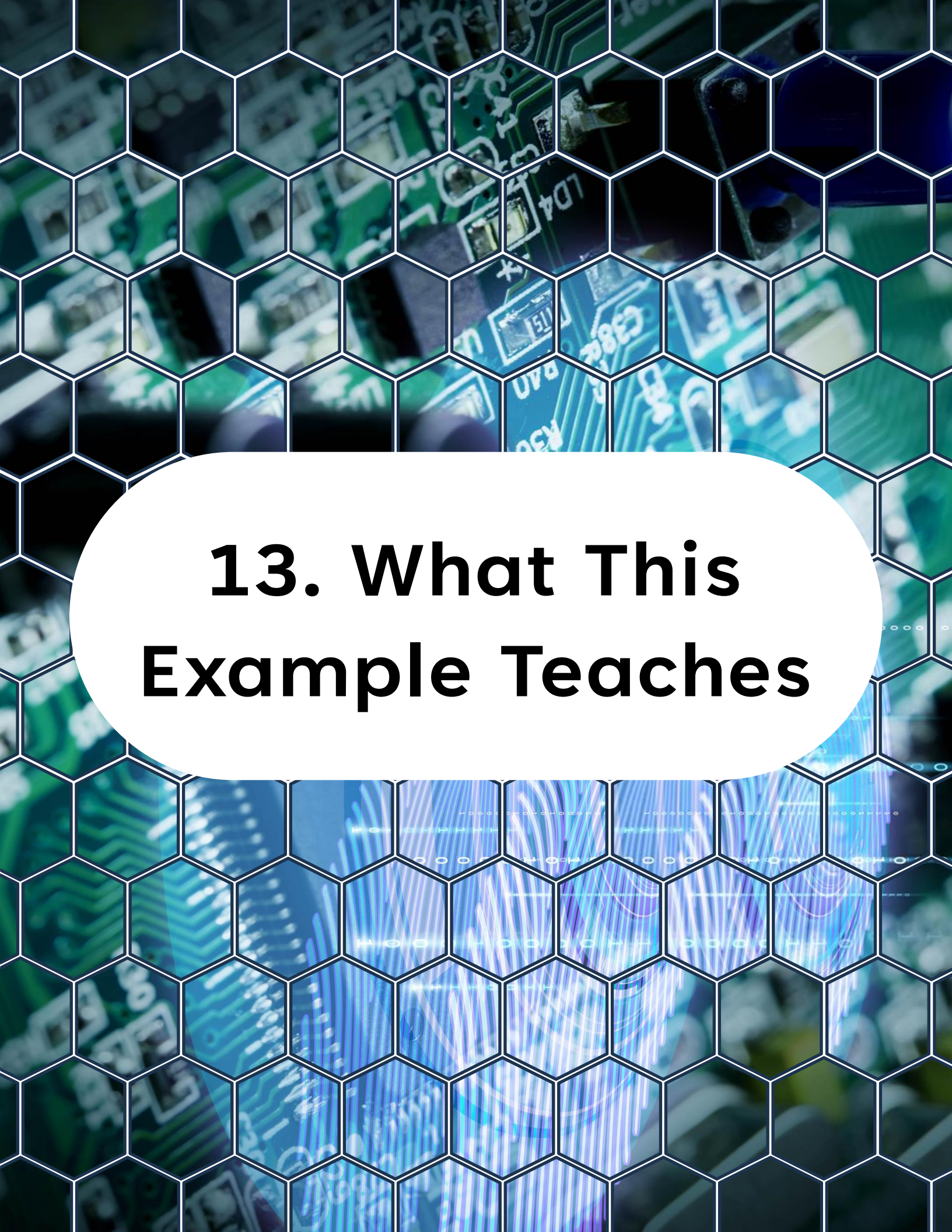
```
156 /**
157  * @brief Task executed every 1000 ms.
158  *
159  * Demonstrates a simple cooperative state machine.
160  */
161 static void task_1000ms(void)
162 {
163     switch (g_status_state)
164     {
165         case STATUS_STATE_IDLE:
166             g_status_state = STATUS_STATE_SAMPLE;
167             break;
168 
169         case STATUS_STATE_SAMPLE:
170             /* Use g_adc_value here if needed */
171             g_status_state = STATUS_STATE_UPDATE;
172             break;
173 
174         case STATUS_STATE_UPDATE:
175             /* Update application status here */
176             g_status_state = STATUS_STATE_IDLE;
177             break;
178 
179         default:
180             g_status_state = STATUS_STATE_IDLE;
181             break;
182     }
183 }
184 
185 /**
186  * @brief TCB0 ISR generates scheduler flags.
187  */
188 ISR(TCB0_INT_vect)
189 {
190     static uint16_t tick_count = 0u;
191 
192     TCB0.INTFLAGS = TCB_CAPT_bm;
193 
194     tick_count++;
```

12. Complete Example - Cooperative Scheduler

```
194     tick_count++;
195
196     if ((tick_count % 10u) == 0u)
197     {
198         task_10ms_flag = 1u;
199     }
200
201     if ((tick_count % 100u) == 0u)
202     {
203         task_100ms_flag = 1u;
204     }
205
206     if ((tick_count % 1000u) == 0u)
207     {
208         task_1000ms_flag = 1u;
209         tick_count = 0u;
210     }
211 }
212
213 /**
214  * @brief Main entry point.
215  *
216  * @return Never returns.
217  */
218 int main(void)
219 {
220
221     // Initialize LED pin
222     LED_init();
223
224     // Initialize ADC0 for periodic sampling
225     ADC0_init();
226
227     // Initialize TCB0 for scheduler tick generation
228     TCB0_init();
229
230     // Enable global interrupts
231     sei();
```

12. Complete Example - Cooperative Scheduler

```
230     // Enable global interrupts
231     sei();
232 
233     while (1)
234     {
235         if (task_10ms_flag)
236         {
237             task_10ms_flag = 0u;
238             task_10ms();
239         }
240 
241         if (task_100ms_flag)
242         {
243             task_100ms_flag = 0u;
244             task_100ms();
245         }
246 
247         if (task_1000ms_flag)
248         {
249             task_1000ms_flag = 0u;
250             task_1000ms();
251         }
252     }
253 }
```



13. What This Example Teaches

13. What This Example Teaches

This example shows a real bare-metal scheduling pattern:

- The timer provides the time base
- The ISR does very little
- The main loop runs the tasks
- Tasks remain short and cooperative
- State machines integrate cleanly into the architecture

This is a very practical structure for small and medium embedded projects.

14. CPU Offloading and Determinism

14. CPU Offloading and Determinism

A cooperative scheduler does not replace hardware offloading.

In a strong design:

- EVSYS handles hardware signaling
- peripherals do hardware work
- the cooperative scheduler handles application-level orchestration

This gives you a clean division:

- hardware handles fast deterministic tasks
- firmware handles logic and sequencing

That is good embedded architecture.

15. Common Mistakes

15. Common Mistakes

This defeats the scheduler.

Bad:

```
task_sensor()
{
    delay_ms(200);
}
```

15.2 Doing Too Much Work in the ISR

The timer ISR should only set flags or maintain counters.

Do not run full application logic there.

15.3 Letting One Task Run Too Long

If one cooperative task takes too long, all other tasks suffer.

Cooperative systems depend on discipline.

15. Common Mistakes

15.4 Ignoring State Machines

As tasks become more complex, state machines keep them manageable.

Without them, cooperative code becomes messy quickly.

16. Practical Lab Ideas

16. Practical Lab Ideas

Try these extensions on your own:

- Add a task that samples a potentiometer every 20 ms
- Add a task that updates a software counter every 1 s
- Add a button task with debouncing every 10 ms
- Convert one task into a proper multi-state state machine
- Compare scheduler behavior with and without blocking delays

These exercises help you feel the difference between structured firmware and ad hoc looping.

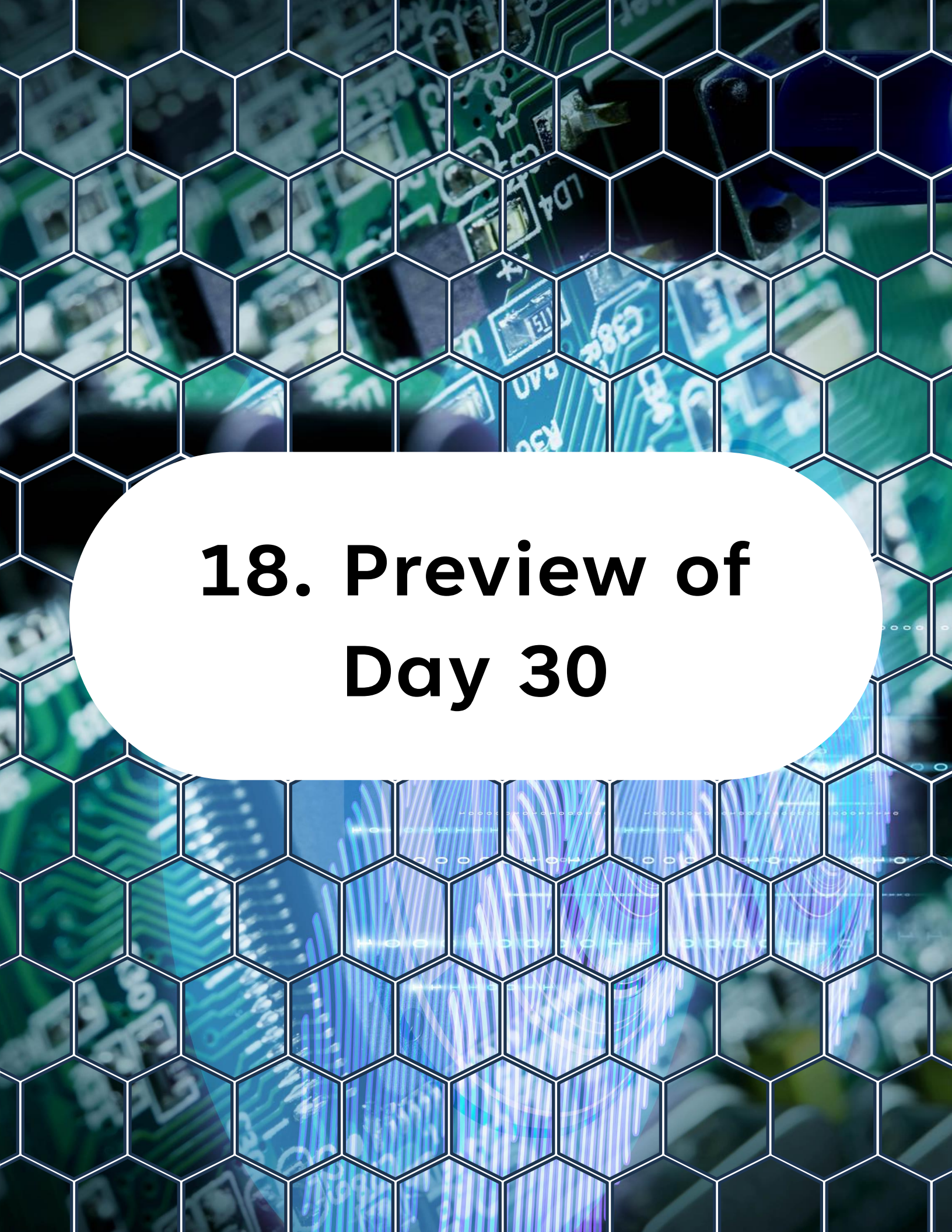
17. What You Should Understand Now

17. What You Should Understand Now

You now know:

- What cooperative scheduling is
- How a super-loop becomes a real scheduler
- How timers provide time slicing
- Why non-blocking tasks matter
- Why state machines fit naturally into this model

This is one of the most useful architectural patterns in bare-metal embedded systems.



18. Preview of Day 30

18. Preview of Day 30

Next

Final Project

We will explore:

- Design a full embedded application
- Implement clean architecture
- Use register-level drivers
- Apply event-driven and cooperative design
- Produce production-quality firmware

This is where learning becomes engineering.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course